影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 3 章 數位影像基礎

本章學習目標

- 說明影像數位化的兩個步驟——取樣與量化，並理解像素與解析度的意義。
- 陳述 Nyquist 取樣定理，解釋混疊現象的成因與防範方式。
- 說明位元深度與動態範圍的關係，理解量化不足造成的假輪廓現象。
- 區分空間解析度、灰階解析度與時間解析度，並判斷各自的取捨。
- 描述影像感測器的成像原理與色彩濾鏡陣列，理解 RAW 與 JPEG 的差異。
- 運用 MSE、PSNR、SSIM 客觀評估影像品質，並理解其侷限。
- 說明像素鄰域、連通性與距離度量，為後續分割與型態學打底。

叫獸開講|第一張數位影像：一位父親、一台巨型電腦與 176 個像素

1957 年初的某一天，一位二十七歲的年輕科學家把一張兒子的照片帶到辦公室。這件事聽起來平凡無奇——多少新手爸媽都做過同樣的事。但他的辦公室不太一樣：那是美國國家標準局(NBS，今日的 NIST)，而他是少數獲准操作 SEAC 的人之一。SEAC 是美國第一台可程式化的儲存程式電腦，大到能塞滿一整個房間，平時的工作是社會安全帳務、空軍後勤運算，以及氫彈相關的計算。

這位科學家名叫 Russell Kirsch。他心裡有一個當時幾乎沒人問過的問題：「如果電腦能夠看見圖片，會發生什麼事?」為了回答它，他和同事打造了一台滾筒式掃描器——照片貼在旋轉的滾筒上，一端是馬達、另一端是閃頻圓盤，搭配一支能偵測極微弱光線的光電倍增管，隨著滾筒緩緩轉動，透過箱壁上一個方形的觀察孔逐點掃描影像的明暗變化，再轉換成電腦看得懂的二進位數字。

他選來測試的，是三個月大的兒子 Walden 的頭肩照。原本那是一張他與兒子的合影，但因為當時的設備最多只能處理 176×176(共 30,976 個)像素，他把照片裁成了 5 公分見方的小方塊——順帶讓兒子的臉，比他自己更有名。掃出來的影像模糊、顆粒粗大、灰階層次極少，像個幽靈，卻是人類史上第一張數位影像。Kirsch 後來自嘲說，他「坦承曾竊取原本該用於更有用產品(比如熱核武器計算)的機器時間」。

這 30,976 個像素，大約只有 2010 年代數位相機一張照片的千分之一資訊量，卻成了往後一切的源頭：衛星影像、電腦斷層掃描、商品條碼、桌上排版、數位攝影，全都從這口井裡湧出。2003 年，《生活》雜誌的編輯把它列入「改變世界的一百張照片」。

故事還有一個耐人尋味的尾聲。Kirsch 晚年回顧時，對自己當年的一個選擇感到不滿意：他把像素做成了方形。他說那是為了實作方便而做的權宜，卻讓全世界沿用了半世紀——方形的網格在表現曲線與斜線時天生笨拙(這也是你在放大影像時看到鋸齒的根本原因，第 4 章的內插正是為了緩解它)。晚年他甚至還在研究以三角形等其他形狀來組成影像。這是一位工程師難得的誠實：承認自己的一個「暫時的決定」變成了整個領域的預設，而它未必是最好的答案。

叫獸把這個故事放在第 3 章開頭，是因為本章要談的正是那個「把連續的世界切成一格一格數字」的過程——取樣與量化。Kirsch 當年受限於設備只能取 176×176、只能表現極少的灰階;今天你的手機能拍下數千萬像素、每個通道 12 位元以上。但本質完全相同：我們永遠是在用有限的格子，去逼近一個無限細緻的真實世界。理解這個「逼近」會遺失什麼、又該怎麼把遺失降到最低，就是本章的全部功課。

思考題：Kirsch 為了實作方便選了方形像素，結果影響了半世紀。回想第 2 章 OpenCV 的 BGR 順序，也是類似的故事。你認為工程師在做「暫時的權宜決定」時，該如何判斷它會不會變成難以更改的長期標準?

3.1 從連續到離散：取樣與量化

3.1.1 影像數位化的兩個步驟

真實世界的影像是連續的：在空間上，兩個點之間永遠可以再找到一個點;在亮度上，兩個灰階之間永遠可以再分出更細的差異。但電腦只能處理有限的數字，因此必須把連續的影像轉換成離散的數值陣列。這個轉換分成兩個獨立的步驟：

1. 取樣(Sampling)：在空間上離散化。把影像切成一格一格的小方塊，每一格取一個代表值——這就是像素(pixel,picture element 的縮寫)。取樣的密度決定了空間解析度，也就是我們常說的「幾百萬畫素」。



圖 3-1　數位化的兩個步驟：取樣切空間，量化切亮度

2. 量化(Quantization)：在亮度上離散化。把每個像素的連續亮度值，對應到有限個階層中的一個。階層數由位元深度決定：8 位元有 256 階(0 至 255)，這正是第 2 章 uint8 的由來。

用一個比喻幫助理解：取樣像是決定「用多細的網格去描摹一幅畫」，量化則是決定「調色盤上有幾種顏色可用」。兩者都會造成資訊損失，而且是不可逆的——一旦取樣或量化完成，失去的細節就永遠回不來了(這也預告了第 7 章影像還原的物理上限)。承叫獸開講，Kirsch 當年在兩個維度上都受到極大限制：176×176 的取樣、極

少的量化階層。



圖 3-2　取樣率由高到低：先消失的是細節

3.1.2 像素、解析度與尺寸的混淆

這裡有一組經常被混用、實則不同的概念，值得釐清：

- 影像尺寸(Image Size)：影像的像素數，例如 1920×1080。這是陣列的形狀(第 2 章的 shape)。
- 空間解析度(Spatial Resolution)：單位長度內的像素數，例如掃描器的 300 dpi(每英寸點數)、感測器的每毫米像素數。它才真正描述「細節的精細程度」。
- 顯示尺寸(Display Size)：影像在螢幕或紙上實際呈現的大小，取決於顯示裝置的密度。

為什麼要區分?因為「像素多」不等於「細節多」。把一張模糊的照片放大四倍存檔，像素數變成十六倍，細節卻一點也沒增加——第 4 章的內插只是在既有資訊之間「猜」出新像素。真正決定細節的，是拍攝當下的光學品質與取樣密度。這也是為什麼手機廠商的「億級畫素」宣傳需要打個問號：感測器再多像素，若鏡頭的解析力跟不上，多出來的像素只是在記錄模糊。



圖 3-3　像素多不等於細節多

3.1.3 三種解析度

完整地說，數位影像其實有三個維度的解析度，各有取捨：



圖 3-4　三種解析度不足時的三種現象

解析度類型	離散化的維度	不足時的現象	相關章節
空間解析度	空間(取樣)	鋸齒、細節喪失、混疊	第 4 章內插、第 6 章混疊
灰階解析度	亮度(量化)	假輪廓、色階斷層	第 5 章灰階轉換、第 11

			章壓縮
時間解析度	時間(影格率)	運動模糊、時間混疊	第 7 章運動模糊、第 12 章視訊

表 3-1 三種解析度與其不足時的現象

三者之間常有取捨關係。以攝影為例：要提高時間解析度(縮短曝光以凍結動作)，進光量就變少，雜訊相對變大;要提高空間解析度(更多像素)，每個像素接收的光就變少，同樣影響訊噪比。這種「此消彼長」的張力，是感測器設計的核心課題，也會在第 5 章夜景模式、第 7 章運動模糊、第 17 章產線取像中反覆出現。

3.2 取樣定理與混疊

3.2.1 Nyquist 取樣定理

取樣要多密才夠?這個問題有一個精確的答案，由 Nyquist 與 Shannon 給出：取樣頻率必須至少是訊號中最高頻率成分的兩倍，才能完整重建原訊號。用影像的語言說：若影像中最細的紋理是每毫米五個明暗週期，取樣密度就必須至少每毫米十個像素。

為什麼是兩倍?直覺上可以這樣理解：要記錄一個完整的波形週期，至少需要兩個取樣點——一個記錄波峰、一個記錄波谷。若每個週期只取到一個點，你完全無從得知這個波的存在，甚至會誤以為它是別的東西。這個「別的東西」，就是下一節的混疊。

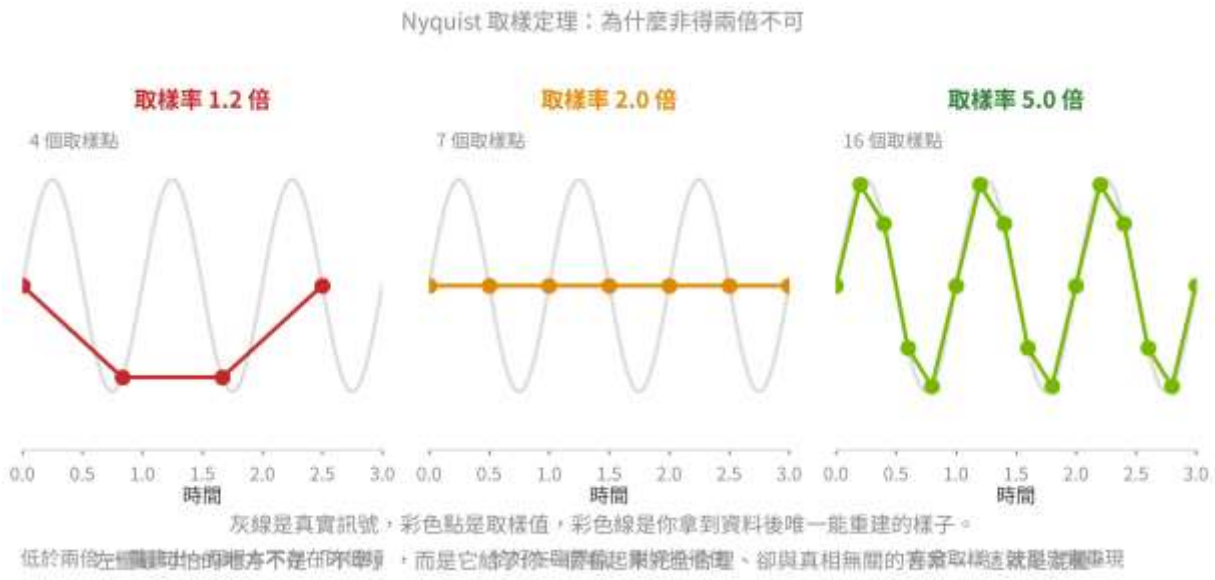


圖 3-5　取樣率 1.2 倍、2 倍、5 倍的重建結果

這條定理的完整意義，要到第 6 章的頻率域才說得清楚：取樣的動作在頻域上會使原訊號的頻譜以取樣頻率為間隔無限複製;取樣夠密時複製品彼此分離、可用低通濾波取回原訊號，不夠密則相鄰複製品重疊、高頻成分「摺疊」進低頻區。第 6.6.1 節會完整展開這個解釋。這裡先記住結論：取樣不足，高頻會偽裝成低頻。

3.2.2 混疊的三張面孔

混疊(Aliasing)是取樣不足的直接後果，它在不同場合有不同的樣貌，但成因完全相同：

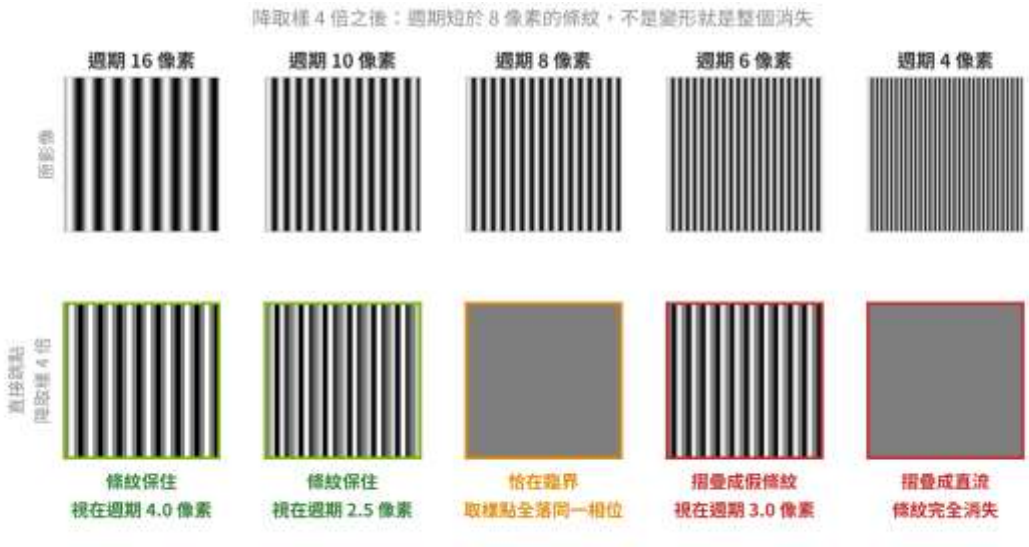


圖 3-6　混疊的分界線：降取樣四倍後的五種條紋

3. 空間混疊——摩爾紋：拍攝細密的條紋(格子襯衫、螢幕、紗窗)時出現的詭異波紋。這是螢幕像素網格與相機感測器網格相互干涉，高頻紋理摺疊成了低頻的粗大波紋。第 6.7.1 節會示範如何以陷波濾波移除它。
4. 鋸齒(Jaggies)：斜線與曲線在低解析度下呈現階梯狀。這是叫獸開講中方形像素的直接後果，也是抗鋸齒(anti-aliasing)技術要對付的問題——其原理正是先做低通模糊，再取樣。
5. 時間混疊——車輪倒轉：影片中高速旋轉的車輪或電風扇看似靜止或倒轉。這是影格率(時間取樣)不足，高頻的旋轉運動被摺疊成了低頻甚至反向的運動。第 12 章叫獸開講的思考題正是這個現象。

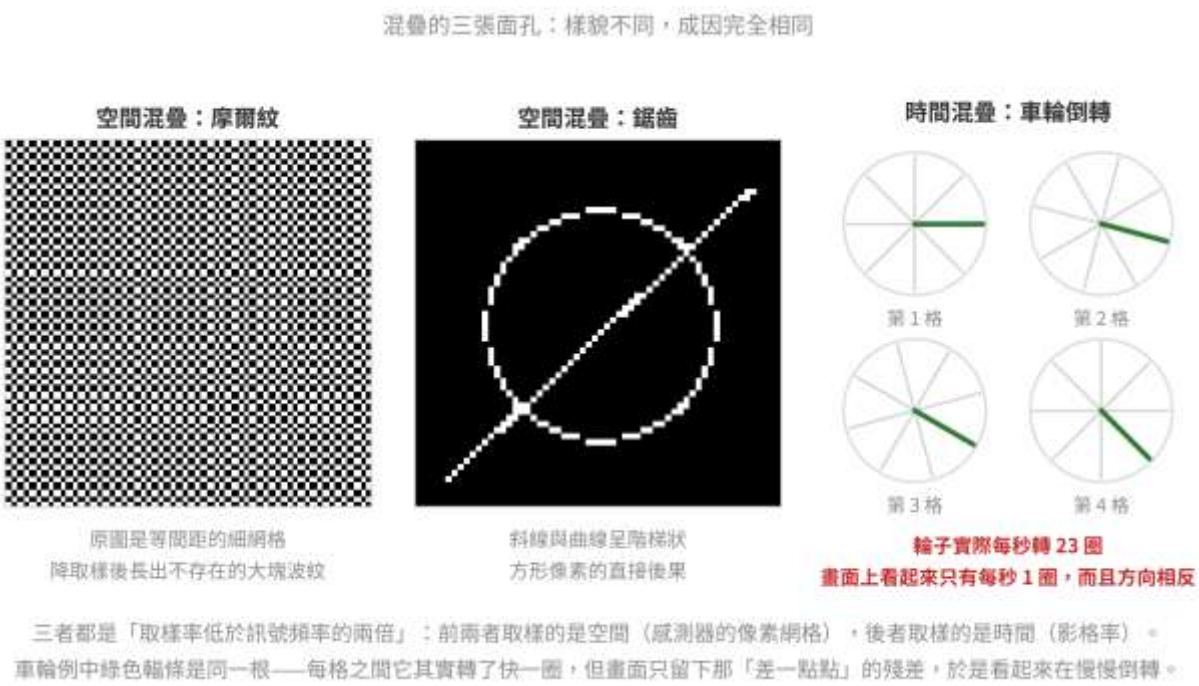


圖 3-7　混疊的三張面孔：摩爾紋、鋸齒、車輪倒轉

3.2.3 如何防範

防範混疊的方法只有一個原則：在取樣之前，先把超過取樣率一半的高頻成分濾掉。這稱為抗混疊濾波(anti-aliasing filter)，硬體上以光學低通濾鏡實現，軟體上則以模糊處理實現。

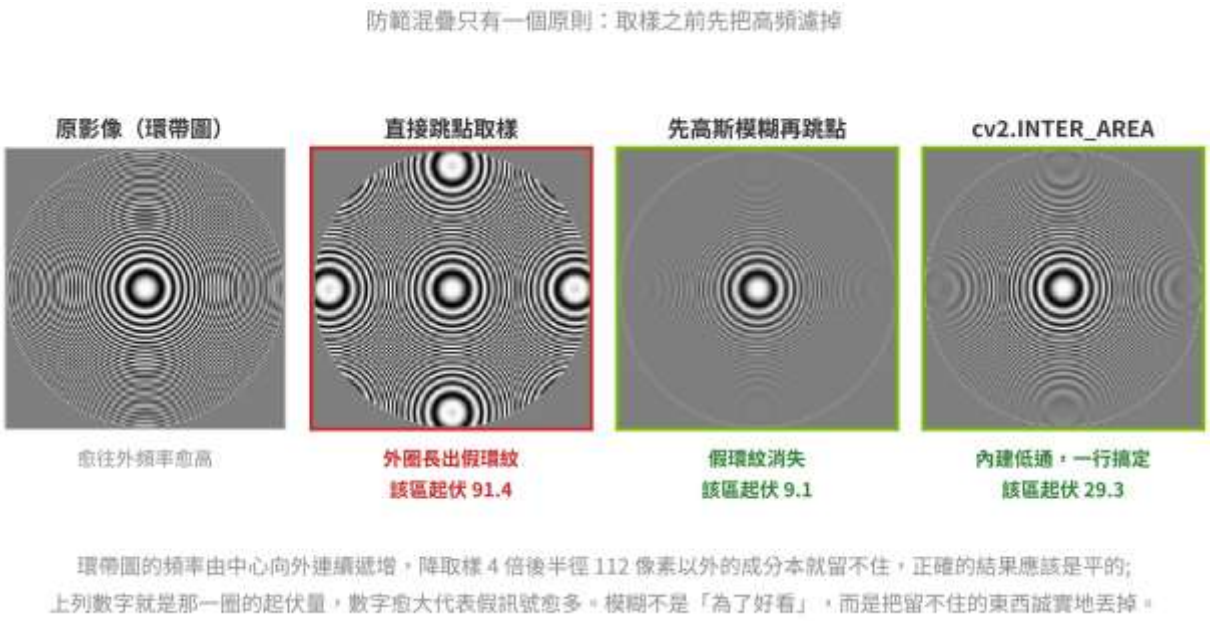


圖 3-8　三種縮圖做法在環帶圖上的實測比較

這解釋了一個常令初學者困惑的建議：縮小影像前應先做模糊。因為縮小就是降低取樣率，若不先把高頻濾掉，它們就會摺疊成假訊號。OpenCV 的 INTER_AREA 內插之所以適合縮小，正是因為它內建了區域平均(即低通)的動作——第 4.3 節的口訣「縮小用 AREA」，根源就在這裡。

程式 3-1 混疊的產生與防範

```
import cv2, numpy as np

# --- 製造混疊:合成高頻條紋後直接降取樣 ---
N = 512
x = np.arange(N)
stripes = ((np.sin(2 * np.pi * x / 4) + 1) * 127).astype(np.uint8)
img = np.tile(stripes, (N, 1))              # 每 4 像素一個週期(高頻)

# 錯誤做法:直接跳點取樣 → 混疊
bad = img[::4, ::4]                          # 條紋消失或變成假圖案

# 正確做法:先低通模糊,再降取樣
blurred = cv2.GaussianBlur(img, (0, 0), sigmaX=2)
good = blurred[::4, ::4]

# OpenCV 的 INTER_AREA 內建低通,是縮小的建議選項(第 4 章)
area = cv2.resize(img, (N//4, N//4), interpolation=cv2.INTER_AREA)

cv2.imwrite("alias_bad.png", bad)
cv2.imwrite("alias_good.png", good)
cv2.imwrite("alias_area.png", area)
# 並排比較:bad 會出現原圖沒有的假圖案
```

3.2.4 推導：取樣在頻率域上究竟做了什麼

3.2.1 節說「這條定理的完整意義，要到第 6 章的頻率域才說得清楚」。這裡先把骨架推一次——它只有四個步驟，而且能讓「摺疊」這個比喻變成字面上的事實。

第一步：取樣這個動作，在數學上等於把連續訊號乘上一列等間距的脈衝。若取樣間距為 Δ，這列脈衝就位於 0、Δ、2Δ、3Δ……的位置，相乘之後只有這些位置的值被留下來，其餘全部歸零。

第二步：依卷積定理（6.5 節），空間域的相乘等於頻率域的卷積。而脈衝列有一個漂亮的性質——它的傅立葉轉換仍然是一列脈衝，只是間距變成了 1/Δ。於是「取樣」在頻率域上就是「原訊號的頻譜與一列間距 1/Δ 的脈衝做卷積」。

第三步：與脈衝列卷積的效果，是把原頻譜原封不動地複製到每一個脈衝的位置。換句話說，取樣讓原本只有一份的頻譜，變成以 1/Δ 為間隔無限延伸的一排複製品。

第四步，也是結論：這些複製品會不會互相重疊，取決於原訊號的最高頻率 f_max 與間隔 1/Δ 的關係。若 f_max < 1/(2Δ)，相鄰的複製品之間留有空隙，彼此分離，此時只要用一個低通濾波器把中央那一份取出來，就能完整重建原訊號——這正是取樣定理。反之若 f_max > 1/(2Δ)，複製品的裙襬互相重疊，高頻的尾巴侵入了鄰居的低頻區，而且重疊之後再也分不開：你無法判斷某個低頻成分究竟是原本就有的，還是從高頻摺疊過來的。

「摺疊」兩個字是字面意義，而且可以直接算。一個頻率為 f 的成分，取樣後會出現在 $|f - k/\Delta|$ 的位置（k 為整數，取使結果最小的那個）。以圖 3-6 週期 6 像素的條紋為例：原頻率為 1/6 週期 / 像素，降取樣 4 倍後在新影像中的頻率是 $4/6 \approx 0.667$，超過了 Nyquist 頻率 0.5，於是摺疊到 $|0.667 - 1| = 0.333$，對應的視在週期是 $1/0.333 = 3$ 像素——這與圖中量到的數字完全一致。

這個推導也一次解釋了三件事。其一，為什麼抗混疊濾波一定要在取樣之前：重

疊一旦發生就不可分離，事後再怎麼濾波都救不回來。其二，為什麼圖 3-6 中週期恰為 8 的條紋也失敗：f_max 恰好等於 1/(2Δ) 時，相鄰複製品剛好相切，理論上仍可分離，但需要一個理想的低通濾波器（第 6.3.2 節會說明它不存在），而且結果對取樣相位極度敏感。其三，為什麼第 6 章要花一整章談頻率域——本章的現象在那裡都只是這張頻譜複製圖的不同切面。

3.3 量化與位元深度

3.3.1 位元深度與動態範圍

位元深度(bit depth)決定每個像素能表示多少個灰階層次：1 位元只有黑與白兩階(二值影像，第 10 章的主角);8 位元有 256 階，是最常見的規格;10、12、14 位元則常見於專業相機的 RAW 檔與醫學影像。彩色影像的位元深度通常以「每通道」計算，8 位元的 RGB 影像共有約一千六百萬種顏色組合。

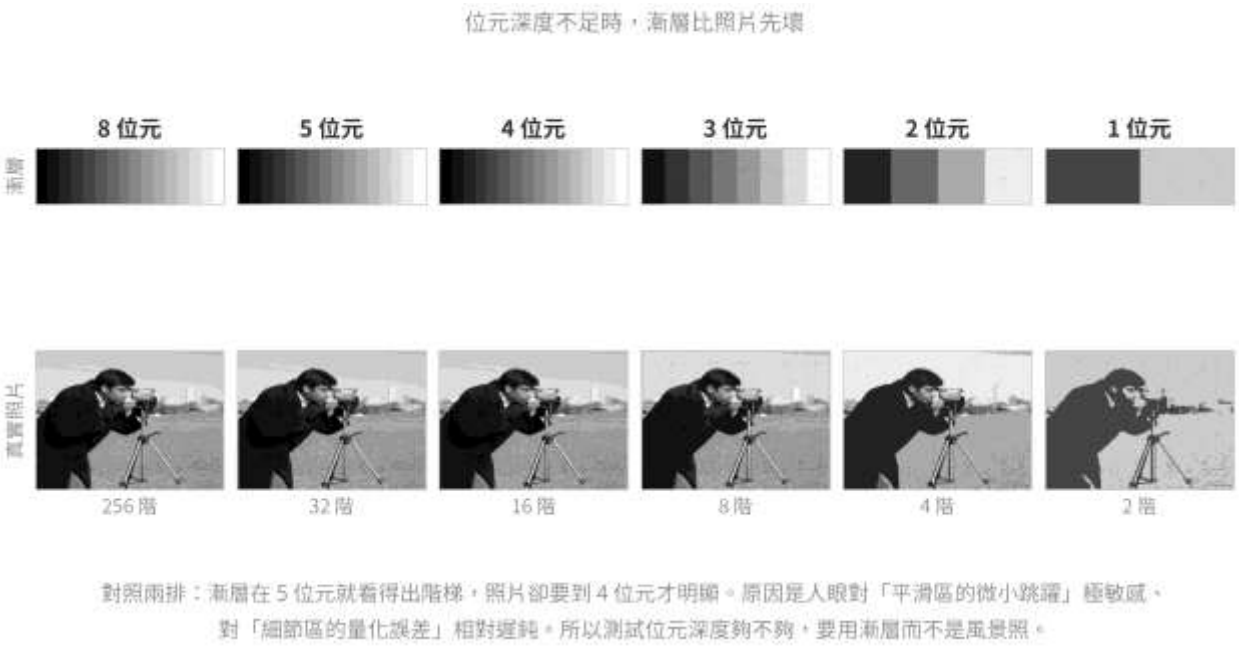


圖 3-9　位元深度 8 到 1：漸層比照片先壞

動態範圍(Dynamic Range)則描述影像能同時記錄的最亮與最暗之間的比值。人眼在單一場景中約可分辨十多個 f-stop 的亮度差異，而一般 8 位元 JPEG 只能記錄較窄的範圍——這就是為什麼逆光拍攝時，若曝光在人臉，背景的天空就會過曝成一片

白。高動態範圍(HDR)技術以不同曝光的多張影像合成，正是為了突破這個限制，與第 5 章夜景模式的多影格合併是同源的思路。

3.3.2 量化不足：假輪廓

當位元深度不足時，最明顯的現象是假輪廓(false contouring，或稱色階斷層、posterization)：在本該平滑漸變的區域(晴朗的天空、柔和的陰影、漸層背景)出現一圈圈生硬的階梯狀邊界。原因很單純：真實的漸變是連續的，但量化後只能用有限的幾階去表示，相鄰階層之間就出現了肉眼可見的跳躍。

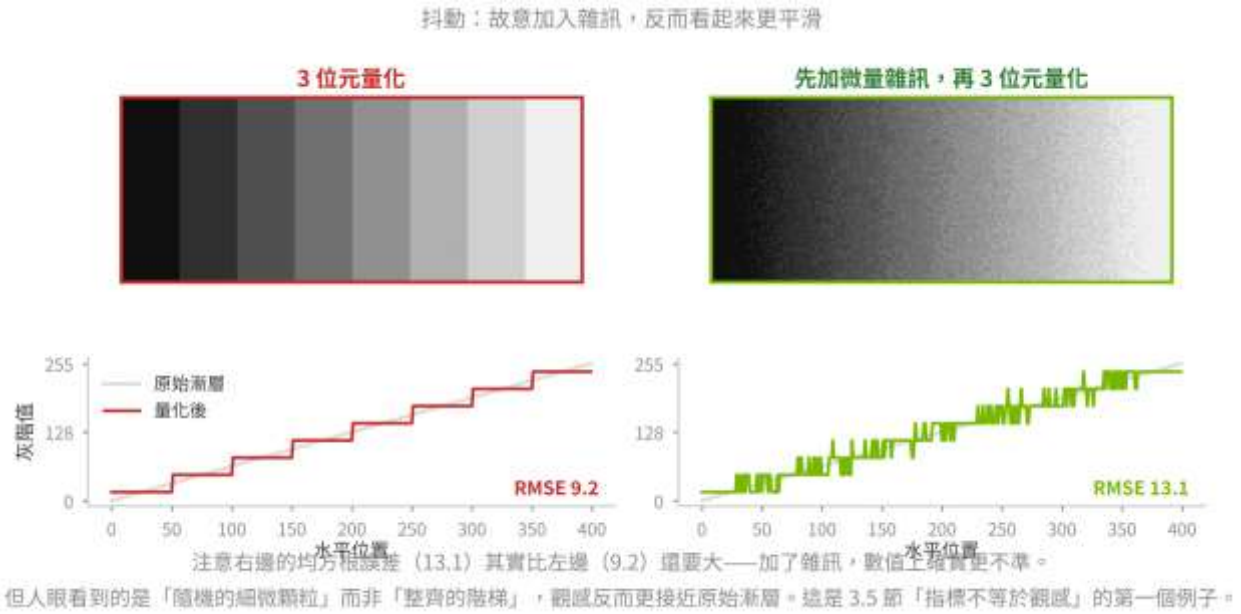


圖 3-11　抖動：加入微量雜訊反而看起來更平滑

有趣的是，人眼對「平滑區域的微小跳躍」特別敏感，對「細節豐富區域的量化誤差」反而遲鈍。這個特性有兩個實務意義：其一，漸層區域是判斷位元深度是否足夠的最佳測試素材;其二，刻意加入極微量的雜訊(稱為抖動，dithering)反而能打散階梯邊界、讓漸變看起來更平滑——這是「用雜訊改善觀感」的少見例子。



圖 3-10　假輪廓為什麼只長在平滑的區域

程式 3-2 量化與假輪廓

```
import cv2, numpy as np

# 建立平滑漸層(最容易看出量化不足的素材)
ramp = np.tile(np.linspace(0, 255, 512, dtype=np.uint8), (200, 1))

# 模擬不同位元深度的量化
for bits in [8, 5, 4, 3, 2, 1]:
    levels = 2 ** bits
    step = 256 // levels
    q = (ramp // step) * step              # 量化
    cv2.imwrite(f"quant_{bits}bit.png", q)
    print(f"{bits} 位元 = {levels} 階")
    # 觀察:位元數愈少,漸層上的階梯(假輪廓)愈明顯

# 抖動:加入微量雜訊打散階梯邊界
noise = np.random.normal(0, 8, ramp.shape)
dithered = np.clip(ramp + noise, 0, 255).astype(np.uint8)
q_dither = (dithered // 32) * 32           # 同樣量化成 3 位元
cv2.imwrite("quant_3bit_dither.png", q_dither)
# 與 quant_3bit.png 比較:加了雜訊反而看起來更平滑
```

3.3.3 量化的一體多面

「用有限的階層逼近連續值」這個思想，會在本書中以三種面貌出現，值得在此先建立連結：本章談的是感測器把光強度量化成灰階;第 11 章 JPEG 壓縮的量化步驟，

是把 DCT 係數除以量化步長取整，丟棄人眼不敏感的高頻;第 17 章的模型量化，則是把神經網路的權重從 32 位元浮點降到 8 位元整數以加速推論。三者的數學核心完全相同——都是在可接受的品質損失下，換取儲存、傳輸或運算的大幅節省。當你在第 11、17 章再次遇到「量化」時，請記得回到本章。

3.3.4 推導：每多一個位元，訊噪比為什麼提升約 6 dB

位元深度與影像品質的關係，可以用一條簡潔的經驗法則概括：每增加一個位元，訊噪比提升約 6 分貝。這條法則在選感測器、決定存檔格式、評估量化影響時都用得上，而它的推導只需要兩步。

第一步，量化誤差有多大。設量化步長為 Δ（8 位元時 $\Delta = 1$，即一個灰階）。四捨五入的量化，其誤差必落在 $-\Delta/2$ 與 $+\Delta/2$ 之間;當訊號本身變化夠豐富時，可以合理地假設誤差在這個區間內均勻分布。均勻分布於 $[-\Delta/2, \Delta/2]$ 的隨機變數，其變異數為 $\Delta^2 / 12$——這是把 e^2 在該區間上積分再除以區間長度的結果，是機率論的標準練習。於是量化雜訊的功率就是 $\Delta^2 / 12$。

第二步，位元數改變時會發生什麼。位元數由 b 增加到 $b + 1$，階層數加倍，同樣的滿刻度範圍被切成兩倍多的階，因此步長 Δ 減半。誤差功率正比於 Δ^2，所以變成原本的四分之一。換算成分貝：$10 \times \log_{10}(4) = 6.02$ 分貝。這就是 6 dB 法則的全部內容——它其實只是「$20 \log_{10} 2$」換個說法。

若把訊號也一併算進去，對一個佔滿整個量化範圍的正弦訊號，可以推出常見的工程公式：$\text{SNR} \approx 6.02b + 1.76$ 分貝。代入幾個數字就很有感覺：8 位元約 50 分貝，12 位元約 74 分貝，14 位元約 86 分貝。由 8 位元升到 12 位元，等於多了 $4 \times 6.02 \approx 24$ 分貝的餘裕——這正是圖 3-14 中 RAW 經得起提亮三倍、JPEG 經不起的量化版本。

兩個必要的提醒。第一，這是理論上限，實務上達不到：感測器的讀取雜訊、暗電流、散粒雜訊（3.4.1 節）都會疊加上去，當它們大於量化雜訊時，再多的位元也不

會改善訊噪比——這就是為什麼手機感測器不會一路堆到 16 位元。一個實用的判準是：量化步長應小於感測器本身的雜訊水準，再小就是在精確地記錄雜訊。第二，均勻分布的假設在訊號幾乎不變時會失效——這正是圖 3-10 的平滑區：該區域的起伏小於一個量化步長，誤差不再隨機，而是整片同向偏移，於是表現為假輪廓而非雜訊。圖 3-11 的抖動之所以有效，正是因為它人為地讓誤差重新變回隨機。

3.4 影像的產生：感測器與色彩

3.4.1 感測器如何看見

數位相機的感測器(CCD 或 CMOS)由數百萬個微小的光電二極體組成，每一個對應一個像素。曝光期間，光子撞擊感光元件產生電子，累積的電荷量正比於接收到的光量;曝光結束後，電荷被讀出、放大、經類比數位轉換器量化成數字。



圖 3-12　感測器的成像鏈與其中兩個不可逆的環節

這個過程解釋了幾個重要現象。其一，光子到達是隨機的，因此即使拍攝完全均勻的畫面，各像素的數值也會有隨機起伏——這就是第 7 章的 Poisson 雜訊(散粒雜訊)，它在低光時特別明顯，也是第 5 章夜景模式要對付的主角。其二，讀出與放大電路本身也會引入雜訊(讀取雜訊)。其三，像素若接收過多光子會「滿溢」，數值卡在

最大值——這就是過曝，一旦發生，該區域的資訊就徹底喪失，任何後製都救不回來。

3.4.2 色彩濾鏡陣列與去馬賽克

這裡有一個常被忽略的事實：感光元件本身只能感測光的強度，不能分辨顏色。那彩色照片是怎麼來的?答案是在感測器表面覆蓋一層色彩濾鏡陣列(CFA)，讓每個像素只接收特定顏色的光。最常見的是拜爾陣列(Bayer pattern)，它以「一紅、兩綠、一藍」為單位重複排列——綠色佔一半，是因為人眼對綠光最敏感(這個設計思想與第 8 章 YCbCr 對色度降取樣、第 11 章 JPEG 的 4:2:0 完全一致：把資源分配給人眼最在意的地方)。



圖 3-13　拜爾陣列與去馬賽克：彩色是濾出來又猜回來的

於是每個像素其實只記錄了三個顏色中的一個，另外兩個必須由鄰近像素「猜」出來——這個過程稱為去馬賽克(demosaicing)，本質上是一種內插(第 4 章)。這也解釋了為什麼相機的彩色細節解析力，實際上低於它的像素數所暗示的水準，以及為什麼在高對比的細節邊緣容易出現偽色。

3.4.3 RAW 與 JPEG：一個關鍵的取捨

相機可以輸出兩種檔案。RAW 保存的是感測器讀出的原始資料——高位元深度(通常 12 至 14 位元)、線性(數值正比於光量)、未經壓縮或僅無損壓縮、未套用白平衡

與銳化。JPEG 則是相機內部已經完成去馬賽克、白平衡、色調曲線、Gamma 編碼、銳化，並以有損壓縮(第 11 章)存成 8 位元的成品。

對影像處理而言，這個差異至關重要，有三個理由：第一，位元深度——12 位元的 RAW 有 4096 階，遠多於 JPEG 的 256 階，後製調整曝光時不易產生假輪廓。第二，線性——RAW 的數值正比於實際光量，而 JPEG 經過 Gamma 編碼後並非線性(第 5.1.2 節詳述)，做精確的光度量測時必須留意。第三，未經破壞——JPEG 的壓縮與銳化都是不可逆的，且反覆存檔會累積失真(第 11.4.2 節)。



圖 3-14　RAW 與 JPEG 各自提亮三倍後的灰階存量

因此本書的建議是：做嚴肅的影像處理與科學量測時，盡可能從 RAW 出發;第 7 章的還原、第 14 至 16 章的模型訓練資料蒐集，都會因為輸入品質而受益。當然，RAW 檔案大、處理慢，一般應用用 JPEG 就夠——這又是一個依需求判斷的工程取捨。

3.5 影像品質的客觀評估

當你用某個演算法處理影像後，如何判斷它「好不好」?主觀的目視評估重要但不可量化、不可重現;因此我們需要客觀指標。這一節建立的工具，將在第 5、6、7、11 章的每一個 Lab 中反覆使用。

3.5.1 MSE 與 PSNR

均方誤差(MSE)是最基本的指標：把處理後影像與參考影像逐像素相減、取平方、再平均。它直接反映「差多少」，值愈小愈好，但它的數值大小與影像的亮度範圍有關，不同影像之間難以比較。

峰值訊噪比(PSNR)以對數形式改良了這一點，單位是分貝(dB)，值愈大愈好。實務上的經驗參考值：超過 40 dB 表示品質極佳、幾乎看不出差異;30 至 40 dB 為良好;20 至 30 dB 可見明顯劣化;低於 20 dB 則品質很差。它是影像壓縮與還原領域最廣泛使用的指標(第 7、11 章)。

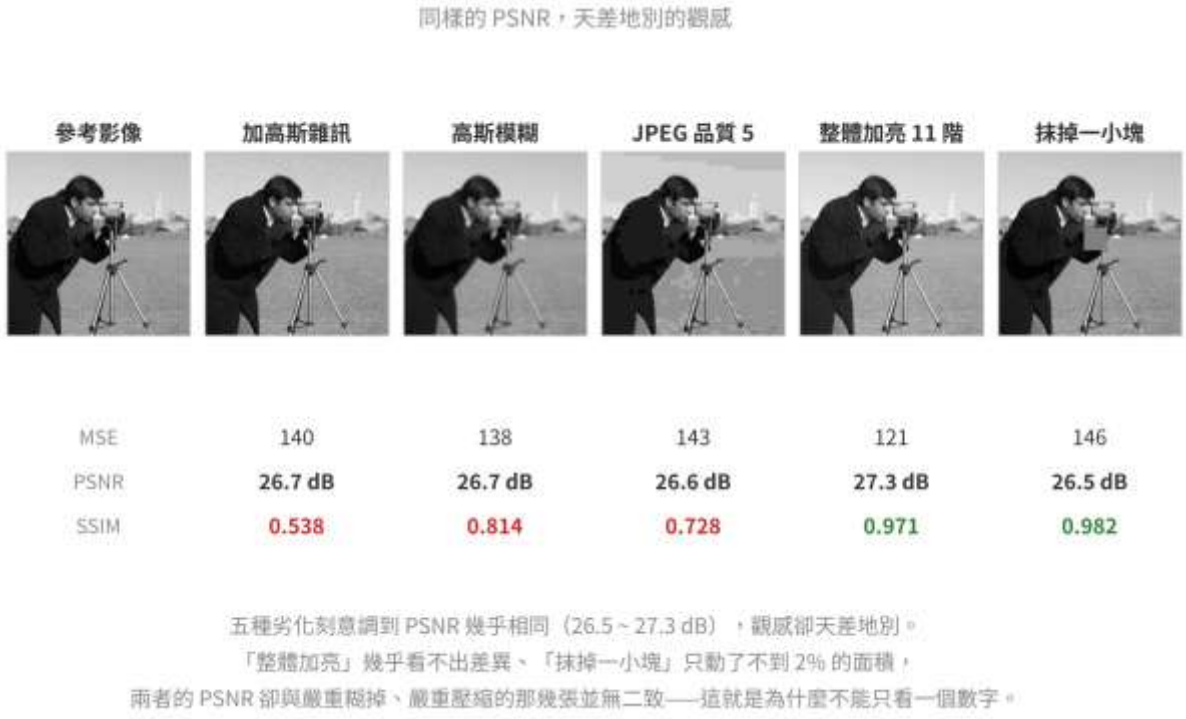


圖 3-15　五種劣化、幾乎相同的 PSNR、天差地別的觀感

3.5.2 SSIM 與 PSNR 的盲點

但 PSNR 有一個根本盲點：它逐像素比較，完全不考慮人類視覺的特性。舉一個經典的例子——把整張影像的亮度平均提高一點點，PSNR 會顯著下降，但人眼幾乎看不出差異;反之，在平坦區域加入少數幾個突兀的雜點，PSNR 下降有限，人眼卻一眼就看到刺眼的瑕疵。換言之，PSNR 高不代表「看起來好」。



圖 3-16　全域指標衡量平均，而你在乎的是最壞的那一塊

結構相似性指標(SSIM)為此而生。它不逐像素比較，而是分別衡量兩張影像在亮度、對比度、結構三個面向的相似程度，再綜合成一個 0 到 1 之間的分數(1 代表完全相同)。因為它著眼於「結構」，更貼近人類的感知。實務建議是兩者並用：PSNR 反映數值誤差，SSIM 反映感知品質，兩者一致時結論可靠，不一致時就值得深究原因。



圖 3-17　SSIM 差異圖：不只「差多少」，還有「差在哪裡」

還有一個更根本的提醒，將在第 7.6.1 節的 MRI 案例中詳述：全域指標衡量的是

平均表現，而許多應用關心的是最壞情況。一個把微小病灶抹掉的去噪演算法，PSNR 可能不降反升(因為背景更乾淨)，但在醫學上是災難。指標是工具，不是目的——這個判斷力，是本書希望你建立的核心素養之一。

程式 3-3 MSE、PSNR 與 SSIM

```
import cv2, numpy as np

clean = cv2.imread("original.png", cv2.IMREAD_GRAYSCALE)
noisy = cv2.imread("processed.png", cv2.IMREAD_GRAYSCALE)

# --- MSE ---
mse = np.mean((clean.astype(np.float64) - noisy.astype(np.float64)) ** 2)
print(f"MSE  = {mse:.2f}")

# --- PSNR(OpenCV 內建) ---
print(f"PSNR = {cv2.PSNR(clean, noisy):.2f} dB")

# --- SSIM(需安裝 scikit-image:pip install scikit-image) ---
from skimage.metrics import structural_similarity as ssim
score, diff_map = ssim(clean, noisy, full=True)
print(f"SSIM = {score:.4f}")

# SSIM 差異圖:可視化「哪裡不像」,除錯很有用
diff_vis = (diff_map * 255).astype(np.uint8)
cv2.imwrite("ssim_map.png", diff_vis)

# --- 示範 PSNR 的盲點 ---
shifted = np.clip(clean.astype(np.int16) + 5, 0, 255).astype(np.uint8)
print(f"整體加亮 5:PSNR={cv2.PSNR(clean, shifted):.2f} dB,"
      f" SSIM={ssim(clean, shifted):.4f}")
# 人眼幾乎看不出差異,但 PSNR 顯著下降 → 指標未必等於觀感
```

3.5.3 推導：PSNR 的那些經驗值從哪裡來

3.5.1 節給了幾個經驗參考值——40 分貝以上極佳、30 到 40 分貝良好。這些數字不是慣例，而是可以算出來的，而且算一次之後你對 PSNR 的理解會完全不同。

由定義出發。均方誤差為 MSE，峰值訊噪比定義為 $\text{PSNR} = 10 \times \log_{10}(\text{MAX}^2 / \text{MSE})$，其中 MAX 是像素的最大可能值，8 位元影像為 255。把常數拆出來：$\text{PSNR} = 20 \times \log_{10}(255) - 10 \times \log_{10}(\text{MSE})$，而 $20 \times \log_{10}(255) = 48.13$。於是對所有 8 位元灰階影像，都有 $\text{PSNR} = 48.13 - 10 \times \log_{10}(\text{MSE})$。

把經驗值代進去反推，意義就具體了。PSNR = 40 分貝對應 MSE = 6.5，換算成

均方根誤差是每個像素平均差 2.5 個灰階——在 256 階中差 2.5 階，人眼確實看不出來。PSNR = 30 分貝對應 MSE = 65，平均差 8.1 個灰階，開始在平滑區域看得出端倪。PSNR = 20 分貝對應 MSE = 650，平均差 25.5 個灰階，那已經是一眼就看得出的劣化。更簡潔的記法是：PSNR 每下降 10 分貝，MSE 增為十倍，平均誤差增為 $\sqrt{10} \approx 3.16$ 倍。

這個推導也揭露了一件常被誤解的事：PSNR 與 MSE 之間是一對一的單調對應，兩者攜帶的資訊完全相同。PSNR 並沒有比 MSE「更懂」影像，它的價值只有兩點——其一，除以 MAX^2 之後數值與影像的動態範圍無關，因此不同影像之間可以比較（這正是 3.5.1 節說 MSE 做不到的事）;其二，取對數之後的尺度比較接近人對「差多少」的感受，也讓數值落在一個好記的範圍。除此之外，MSE 的所有盲點，PSNR 一個也沒少——圖 3-15 與 3-16 示範的正是這一點。

最後補一個實務細節：比較不同論文或不同工具的 PSNR 數字時要小心，因為計算方式可能不同。彩色影像有人先轉灰階再算、有人三個通道各自算再平均、也有人只算 Y 通道（第 8 章）;有人把邊界幾個像素排除、有人不排除。這些差別足以造成一兩分貝的落差，而一兩分貝在論文的比較表上往往就是勝負。自己做實驗時，請把計算方式明確寫下來——這是 2.5.4 節「可重現」的具體實踐。

3.6 像素的鄰域、連通性與距離

本節建立的三個概念看似瑣碎，卻是第 9 章分割、第 10 章型態學與連通元件標記的直接基礎。

3.6.1 鄰域與連通性

一個像素的「鄰居」有兩種定義方式：4-鄰域只算上下左右四個像素;8-鄰域則加上四個對角，共八個。由此衍生出兩種連通性，而選擇哪一種會直接影響結果——這在第 10 章計算「畫面中有幾個物件」時尤其關鍵。



圖 3-18　4-鄰域、8-鄰域，與對角相接的經典困境

舉一個經典的困境：兩個只在對角相接的白色區塊，在 8-連通下算一個物件，在 4-連通下算兩個;而一條斜向的細線，在 4-連通下會被判定為斷裂的一群孤立點。更麻煩的是所謂的連通性悖論——若前景與背景採用同一種連通性，會出現「一條封閉的曲線卻沒有把背景分成內外兩部分」這類矛盾。實務上的慣例是前景與背景採用互補的連通性(例如前景 8-連通、背景 4-連通)。

3.6.2 距離度量

兩個像素有多遠?三種常見的定義各有用途：歐氏距離(直線距離)最符合直覺、也最常用於量測;城市街區距離(D4，只能上下左右移動的步數)與棋盤距離(D8，可斜走的步數)則計算快速，且與 4-連通、8-連通對應。

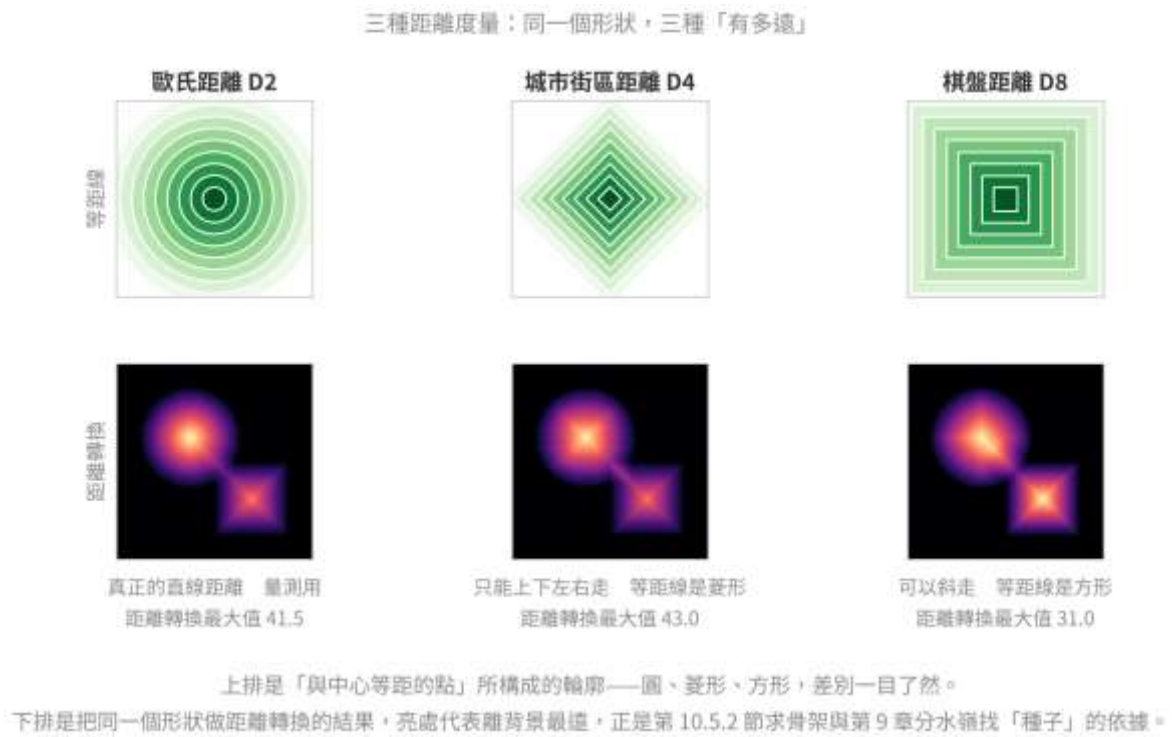


圖 3-19　三種距離度量的等距線與距離轉換

這些度量在第 10.5.2 節的距離轉換中扮演核心角色：為每個前景像素計算它到最近背景像素的距離，得到的「距離圖」可用於求物件骨架，更是第 9 章分水嶺分割「分開相黏物件」的關鍵——每個相黏圓形物件的中心會在距離圖上形成一個局部極大值，正好作為分水嶺的標記。屆時請記得回到本節。

程式 3-4 連通性與距離度量

```
import cv2, numpy as np

# 建立一個測試二值影像:兩個對角相接的方塊
img = np.zeros((100, 100), np.uint8)
img[20:50, 20:50] = 255
img[50:80, 50:80] = 255            # 與前者僅在一個角相接

# 連通性的影響:同一張圖,不同答案
n4, _ = cv2.connectedComponents(img, connectivity=4)
n8, _ = cv2.connectedComponents(img, connectivity=8)
print(f"4-連通:{n4-1} 個物件")     # 2(視為分開)
print(f"8-連通:{n8-1} 個物件")     # 1(視為相連)

# 三種距離度量的距離轉換(第 10 章)
for name, dist_type in [("歐氏", cv2.DIST_L2),
                        ("城市街區", cv2.DIST_L1),
                        ("棋盤", cv2.DIST_C)]:
    d = cv2.distanceTransform(img, dist_type, 5)
```

```
    print(f"{name}距離:最大值 = {d.max():.1f}")
    vis = cv2.normalize(d, None, 0, 255, cv2.NORM_MINMAX)
    cv2.imwrite(f"dist_{dist_type}.png", vis.astype(np.uint8))
```

3.7 應用案例

3.7.1 案例一：掃描解析度該設多少

情境：要把一批紙本文件或老照片數位化典藏，掃描解析度該設 150、300 還是 600 dpi?這是取樣定理最貼近生活的應用。判斷依據是「你要保留的最細特徵有多細」：一般文字文件的筆畫寬度約 0.2 至 0.3 毫米，依取樣定理需要每毫米至少八到十個取樣點，對應約 200 至 300 dpi;若要保留印刷網點或做後續的文字辨識(OCR)，則建議 300 至 600 dpi;而老照片的細節(如髮絲、織品紋理)則可能需要更高。

反過來說，盲目提高解析度也有代價：檔案大小以平方成長(600 dpi 是 300 dpi 的四倍大)、掃描時間拉長、且若原稿本身的細節有限，多出來的像素只是在記錄底片顆粒或紙張纖維。這個「依需求決定規格」的思考，正是第 17 章延遲預算與第 18 章專題規劃的同一種能力。

3.7.2 案例二：醫學影像為什麼需要高位元深度

情境：CT 與 MRI 影像通常以 12 至 16 位元儲存，遠高於一般照片的 8 位元。為什麼?因為人體組織之間的密度差異可能極小——一個早期的腫瘤與周圍正常組織的訊號差，可能只有幾個灰階單位。若量化成 256 階，這個差異可能被歸進同一階而完全消失。

但這帶來一個顯示的難題：一般螢幕只能顯示 256 階灰度，人眼也分辨不了那麼多階。解法是「視窗化」(windowing)：醫師依觀察目標，把 12 位元資料中的某一段範圍拉伸到完整的顯示範圍——看肺部用一組視窗、看骨骼用另一組、看軟組織再一組。同一份資料，不同的視窗設定會看到截然不同的內容。這其實就是第 5 章灰階轉換的醫學版本，也說明了為什麼高位元深度的原始資料必須完整保存：它承載了顯示時尚未取用的資訊。

3.7.3 案例三：工業檢測的取像規格計算

情境：要檢測一個工件表面上寬度 0.1 毫米的刮痕，相機該選多少像素?這是產線設計的第一道題，而它是可以精確計算的。

計算方式：依取樣定理，要可靠地偵測 0.1 毫米的特徵，至少需要每 0.05 毫米一個像素(兩倍取樣);實務上為了穩健，常要求三到五個像素涵蓋最小特徵，即每像素約 0.02 至 0.03 毫米。若工件的視野寬度是 60 毫米，則所需的水平像素數約為 60 除以 0.025，即 2400 像素。加上安全餘裕，選一台 3000 萬像素等級的相機或縮小視野分次拍攝。



圖 3-20　由最小特徵推算相機規格的四個步驟

這個案例展示了工程與「調參數」的差別：規格不是試出來的，而是從需求推算出來的。同樣的推算思維會出現在第 7.6.3 節(由產線速度與曝光時間推算運動模糊長度)與第 17.4.2 節(由機器人速度推算延遲預算)。這種「從需求反推規格」的能力，是叫獸最希望你從這門課帶走的東西之一。

3.7.4 案例四：影像壓縮的品質決策

情境：網站要上傳數千張商品照片，JPEG 品質參數該設多少?這是 3.5 節評估指標的實戰。做法：取一組代表性的照片，分別以品質 95、85、75、60、40 存檔，記

錄每個設定的檔案大小、PSNR 與 SSIM，繪製「檔案大小對品質」的曲線(即第 11 章的率失真曲線)，找出「再降就明顯劣化」的轉折點。

經驗上，品質 85 附近往往是甜蜜點——檔案大小比 95 小得多，而肉眼幾乎看不出差異。但這個結論會因影像內容而異：含大面積平滑漸層的照片對壓縮較敏感(容易出現第 11 章的方塊效應)，而紋理豐富的照片則較耐壓。這也再次呼應 3.5.2 節：指標要看，眼睛也要看。

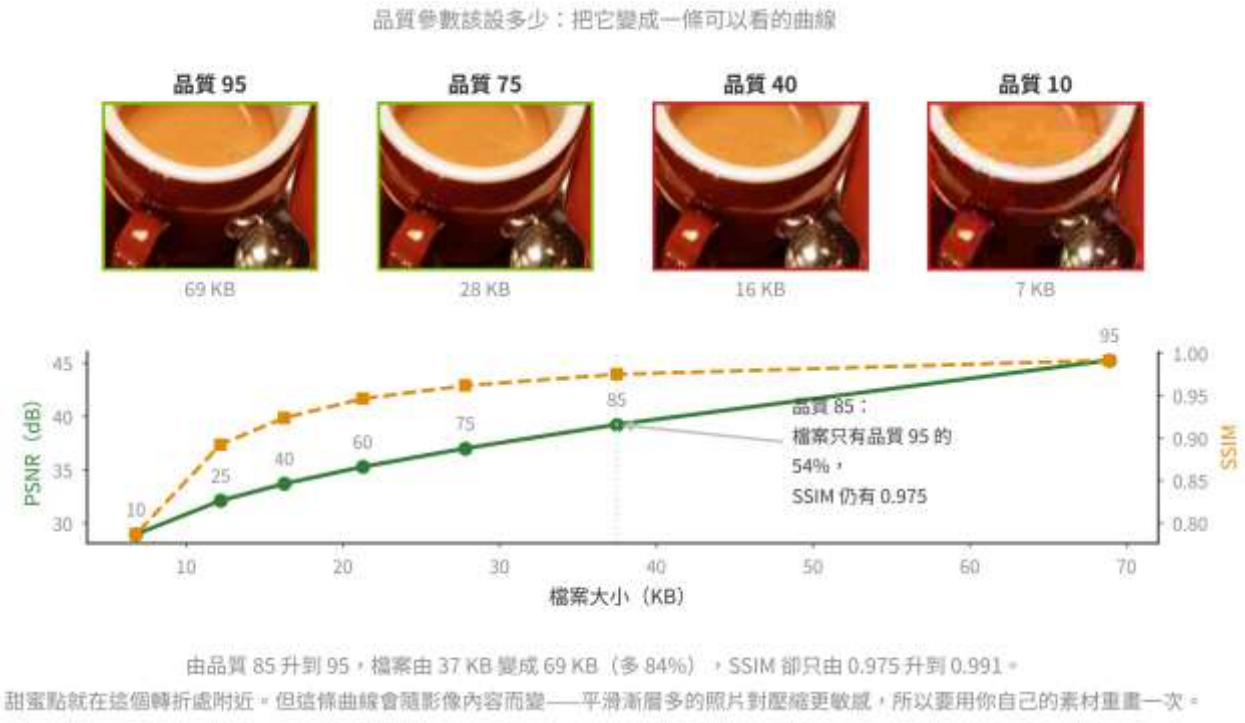


圖 3-21　JPEG 品質參數掃描：檔案大小與品質的取捨曲線

3.8 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
取樣與內插	第 4 章	縮放的內插法選擇;縮小用 INTER_AREA 防混疊
取樣定理與混疊	第 6、12 章	頻域解釋(頻譜複製與摺疊);摩爾紋移除;時間混疊
量化	第 11、17 章	JPEG 的 DCT 係數量化;神經網路的模

		型量化
位元深度與動態範圍	第 5、7 章	Gamma 與對數轉換;HDR;RAW 為還原的理想輸入
感測器雜訊	第 5、7 章	Poisson 與讀取雜訊;夜景多影格平均;去雜訊方法
拜爾陣列與去馬賽克	第 4、8、11 章	內插;人眼對綠光敏感;色度降取樣的同源思想
MSE、PSNR、SSIM	第 5-7、11、14 章	所有處理成效的客觀驗收;全域指標的盲點
鄰域與連通性	第 9、10 章	連通元件標記的計數依據;型態學的結構元素
距離度量	第 9、10 章	距離轉換;分水嶺標記;骨架化

表 3-2 第 3 章與全書其他章節的觀念連結

程式實作 Lab 3

Lab 3-1 取樣、混疊與抗混疊

目標：親眼見證取樣定理。(a)以 np.sin 合成一組不同疏密的條紋影像(每 2、4、8、16 像素一個週期);(b)對每張直接跳點降取樣四倍，觀察哪些出現了「原圖沒有的假圖案」，並用取樣定理解釋為什麼;(c)改為先高斯模糊再降取樣，比較結果;(d)用 OpenCV 的四種內插旗標(NEAREST、LINEAR、CUBIC、AREA)各縮小一次，比較混疊程度，驗證第 4 章「縮小用 AREA」的建議;(e)實拍：用手機拍攝格子襯衫、紗窗或電腦螢幕，重現摩爾紋，並保存這張影像——第 6 章 Lab 會用它來做陷波濾波移除。

Lab 3-2 量化與假輪廓

目標：理解位元深度的意義。(a)以程式 3-2 對一張平滑漸層與一張真實照片分別做 8、5、4、3、2、1 位元的量化，並排比較;(b)記錄「從幾位元開始肉眼可見假輪

廓」，並比較漸層與紋理豐富的照片，何者較早出現、為什麼;(c)實作抖動：量化前加入微量高斯雜訊，比較有無抖動的視覺差異;(d)量化計算 PSNR 與 SSIM 隨位元數的變化，繪成曲線;(e)討論：為什麼醫學影像需要 12 位元以上?(對照 3.7.2 節)

Lab 3-3 品質評估工具箱

目標：建立整學期都會用到的評估工具。(a)寫一個函式 evaluate(original, processed)，一次回傳 MSE、PSNR、SSIM 三個指標，並輸出 SSIM 差異圖;(b)驗證 PSNR 的盲點：對同一張影像做兩種處理——整體亮度加 5、以及在平坦區加入十個突兀的雜點，比較兩者的 PSNR 與 SSIM，並用肉眼判斷哪個比較難看;(c)討論結果與你的視覺判斷是否一致，不一致的原因是什麼;(d)把這個函式存成共用模組(如 utils/metrics.py)，往後第 5、6、7、11 章的 Lab 直接引用——呼應第 2 章「不要重造輪子」。

Lab 3-4 連通性與距離轉換

目標：為第 9、10 章打底。(a)以程式 3-4 建立「對角相接的兩個方塊」，分別用 4-連通與 8-連通做連通元件標記，確認計數結果不同;(b)畫一條斜向的一像素細線，觀察 4-連通下它是否被判為斷裂;(c)對一張含多個圓形物件(可用 cv2.circle 合成，刻意讓兩個相接)的二值影像做距離轉換，把結果以第 8 章的虛擬色彩顯示;(d)找出距離轉換的局部極大值，觀察它們是否落在每個圓的中心——這正是第 9 章分水嶺分割的標記來源;(e)比較歐氏、城市街區、棋盤三種距離度量的結果差異。

Lab 3-5 把混疊做出來，再把它修好

目標：讓取樣定理從一條公式變成你親手量過的數字。(a)用 np.sin 合成週期為 16、10、8、6、4 像素的條紋影像各一張，直接跳點降取樣 4 倍，並排顯示，標出哪幾張失真;(b)對每一張計算「降取樣後的視在週期」，與圖 3-6 的公式對照，確認你算出的數字與看到的現象一致;(c)重做一次，但這次先以不同 sigma 的高斯模糊再降取樣，找出「剛好能消除假圖案」的最小 sigma，並解釋它為什麼是這個值;(d)合成一張環帶

圖（提示：亮度為 $\cos(\pi r^2/k)$ 的形式），分別以直接跳點、先模糊、cv2.INTER_AREA、cv2.INTER_NEAREST 縮小四倍，量測臨界半徑以外區域的標準差，排出優劣;(e)拿一張你自己拍的、含細密紋理的照片（格子衣服、紗窗、遠處磁磚）重做 (d)，確認結論在真實影像上也成立;(f)寫下結論：你會把哪一種縮圖方法設成專案的預設?為什麼?

Lab 3-6 品質指標的極限

目標：親手做出「指標說好、眼睛說壞」的反例，並建立整學期都會用到的評估工具。(a)寫一個函式 evaluate(ref, test)，一次回傳 MSE、PSNR、SSIM 與 SSIM 差異圖;(b)對同一張參考影像製造五種劣化（雜訊、模糊、重度 JPEG、整體加亮、抹掉一小塊），調整各自的參數，讓五者的 PSNR 落在 1 dB 之內——這一步本身就很有教育意義，你會發現要湊到一樣有多容易;(c)把五張圖與三個指標做成一張對照表，請三位同學在不看數字的情況下排序「哪張看起來最糟」，再與指標的排序比較;(d)針對「抹掉一小塊」，試著設計一個能抓出它的指標（提示：考慮局部 SSIM 的最小值、或誤差最大的前 1% 區域），驗證它確實能把這個案例排到最後;(e)以你自己的照片重做一次，確認你設計的指標沒有過度擬合到課本的範例影像;(f)把 evaluate 放進第 2 章 Lab 2-6 的工具箱。第 5、6、7、11 章的每一次「這樣有沒有比較好」都會用到它。

本章重點整理

- 影像數位化分為取樣(空間離散化，決定像素與空間解析度)與量化(亮度離散化，決定位元深度);兩者皆造成不可逆的資訊損失。
- 影像尺寸、空間解析度、顯示尺寸三者不同;像素多不等於細節多——放大既有影像只是內插猜值，並未增加資訊。
- Nyquist 取樣定理要求取樣頻率至少為最高頻率的兩倍;不足則產生混疊，表現為摩爾紋(空間)、鋸齒、車輪倒轉(時間)。防範之道是取樣前先低通濾波，故縮小影像應先模糊或用 INTER_AREA。

- 位元深度決定灰階層次(8 位元 256 階)，不足時在平滑區產生假輪廓;動態範圍描述可同時記錄的明暗比值;加入微量雜訊(抖動)反而能打散階梯。量化思想在第 11 章 JPEG 與第 17 章模型量化中重現。
- 感測器以光電效應成像，帶來 Poisson 與讀取雜訊、以及過曝的不可逆資訊喪失;拜爾陣列使每像素僅記錄一個顏色，需經去馬賽克內插;RAW 保有高位元深度與線性數值，是嚴肅影像處理的理想輸入。
- 客觀品質評估：MSE(誤差平方均值)、PSNR(對數形式，40 dB 以上極佳)、SSIM(衡量亮度、對比、結構，更貼近感知);PSNR 有逐像素比較的盲點，全域指標亦掩蓋最壞情況，須與目視並用。
- 鄰域分 4-鄰域與 8-鄰域，連通性的選擇直接影響物件計數;距離度量有歐氏、城市街區、棋盤三種，為第 10 章距離轉換與第 9 章分水嶺的基礎。

習題

1. 影像數位化的兩個步驟是什麼?各自離散化了哪一個維度?為什麼說這個過程不可逆?

   提示與參考解答：取樣：在空間上離散化，把影像切成一格格像素，決定空間解析度。量化：在亮度上離散化，把連續亮度對應到有限階層，由位元深度決定。不可逆是因為兩者都丟棄了原始的連續資訊——落在同一格內的空間細節、落在同一階層內的亮度差異，都永遠無法還原。這也是第 7 章影像還原的物理上限。

2. 請區分影像尺寸、空間解析度與顯示尺寸。為什麼說「像素多不等於細節多」?

   提示與參考解答：影像尺寸是像素數(如 1920×1080，即陣列的 shape);空間解析度是單位長度內的像素數(如 300 dpi)，才真正描述細節精細程度;顯示尺寸是實際呈現的大小，取決於顯示裝置密度。像素多不等於細節多：把模糊照片放大四倍，像素變十六倍但細節未增(內插只是猜值);感測器像素再多，若鏡頭解析力不足，多出的像素只是記錄模糊。

3. 請陳述 Nyquist 取樣定理，並用「記錄一個波形週期」的直覺解釋為什麼是兩倍。

提示與參考解答：取樣頻率必須至少是訊號中最高頻率成分的兩倍，才能完整重建原訊號。直覺解釋：要記錄一個完整的波形週期，至少需要兩個取樣點——一個抓波峰、一個抓波谷;若每週期只取到一個點，就無從得知這個波的存在，甚至會誤判成別的低頻訊號(即混疊)。

4. 混疊在空間與時間上分別表現為什麼現象?請各舉一例並說明成因相同之處。

提示與參考解答：空間混疊：摩爾紋(拍格子襯衫或螢幕時的詭異波紋)、鋸齒(斜線呈階梯狀)。時間混疊：車輪或電風扇在影片中看似靜止或倒轉。成因相同：取樣率(空間的像素密度或時間的影格率)不足以滿足取樣定理，高頻成分被「摺疊」進低頻區，偽裝成原本不存在的低頻訊號。

5. 為什麼縮小影像前應先做模糊?這與 OpenCV 的 INTER_AREA 有何關聯?

提示與參考解答：縮小即降低取樣率，依取樣定理，超過新取樣率一半的高頻成分會摺疊成混疊(假圖案)。先低通模糊把這些高頻濾掉，就無物可摺疊。INTER_AREA 內插的原理是區域平均，本身就內建了低通動作，因此是縮小的建議選項(第 4.3 節「縮小用 AREA」的根源)。

6. 什麼是假輪廓?它為什麼最容易出現在平滑漸層區域?為什麼加入微量雜訊反而能改善?

提示與參考解答：假輪廓是位元深度不足時，在本該平滑漸變的區域出現生硬的階梯狀邊界。最容易出現在漸層區，是因為真實漸變是連續的，量化後只能用有限階層表示，相鄰階層間出現肉眼可見的跳躍;而人眼對平滑區的微小跳躍特別敏感。加入微量雜訊(抖動)能打散規則的階梯邊界，使量化誤差在視覺上呈隨機分布而非整齊的界線，反而看起來更平滑。

7. 本書中「量化」這個思想出現在哪三個地方?它們的共通核心是什麼?

提示與參考解答：一、本章：感測器把連續的光強度量化成有限灰階。二、第 11 章：JPEG 把 DCT 係數除以量化步長取整，丟棄人眼不敏感的高頻。三、第 17 章：模型量化把神經網路權重從 32 位元浮點降到 8 位元整數。共通核心：在可接受的品質損失下，用有限的階層逼近連續值，換取儲存、傳輸或運算的大幅節省。

8. 感光元件本身能分辨顏色嗎?彩色照片是如何產生的?拜爾陣列為什麼綠色佔一半?

提示與參考解答：不能，感光元件只能感測光的強度。彩色是靠感測器表面的色彩濾鏡陣列(CFA)，讓每像素只接收特定顏色的光，再以去馬賽克內插補出另外兩色。拜爾陣列以「一紅兩綠一藍」重複排列，綠色佔一半是因為人眼對綠光最敏感——這與第 8 章 YCbCr 對色度降取樣、第 11 章 JPEG 的 4:2:0 是同一個設計思想：把資源分配給人眼最在意的地方。

9. RAW 與 JPEG 有哪三個關鍵差異?為什麼做嚴肅的影像處理時建議從 RAW 出發?

提示與參考解答：一、位元深度：RAW 通常 12 至 14 位元(4096 階以上),JPEG 僅 8 位元(256 階)，後製調整不易產生假輪廓。二、線性：RAW 數值正比於實際光量，JPEG 經 Gamma 編碼後非線性(第 5.1.2 節)，精確光度量測須留意。三、未經破壞：JPEG 已套用白平衡、銳化並有損壓縮，皆不可逆且反覆存檔會累積失真。因此第 7 章還原與模型訓練資料蒐集都會因 RAW 的輸入品質而受益。

10. 請說明 PSNR 的盲點，並解釋 SSIM 如何改善。為什麼說「指標是工具，不是目的」?

提示與參考解答：PSNR 逐像素比較、不考慮人類視覺特性：整張影像亮度微幅提高會使 PSNR 顯著下降但人眼幾乎無感;平坦區的少數突兀雜點對 PSNR 影響有限卻極為刺眼。SSIM 改為衡量亮度、對比度、結構三面向的相似度，更貼近感知。「指標是工具不是目的」：全域指標衡量平均表現，而許多應用關心最壞情況——一個抹掉微小病灶的去噪演算法，PSNR 可能不降反升(第 7.6.1 節 MRI 案例)，但在醫學上是災難。

11. 4-連通與 8-連通有何差異?請舉一個「同一張影像、不同連通性得到不同物件數」的例子，並說明為什麼實務上前景與背景要用互補的連通性。

提示與參考解答：4-鄰域只算上下左右，8-鄰域再加四個對角。例子：兩個僅在對角相接的白色方塊，8-連通視為一個物件、4-連通視為兩個;一條斜向細線在 4-連通下會被判為斷裂的孤立點。前景與背景用互補連通性(如前景 8、背景 4)是為了避免連通性悖論——若兩者同用一種，會出現「封閉曲線卻沒把背景分成內外」這類矛盾。

12. (綜合題)承 3.7.3 節：要檢測工件上寬度 0.2 毫米的裂紋，視野寬度為 100 毫米，請推算所需的相機水平像素數，並說明你的假設與安全餘裕。

提示與參考解答：依取樣定理，偵測 0.2 毫米特徵至少需每 0.1 毫米一像素(兩倍取樣);但實務上為穩健，常要求三到五像素涵蓋最小特徵，取四像素則每像素約 0.05 毫米。所需水平像素數 = 100 ÷ 0.05 = 2000 像素。加上安全餘裕(考慮鏡頭邊緣解析力下降、對焦誤差、工件擺放偏移)，建議選 2500 至 3000 像素以上的相機，或縮小視野分次拍攝。關鍵在於說明假設(每特徵幾個像素)與餘裕的理由，而非單一數字。

13. 計算題：承圖 3-6。一張影像中有週期分別為 16、10、8、6、4 像素的條紋，若直接跳點降取樣 4 倍，哪些會失真?請算出分界線，並說明週期恰為 8 的那一組為什麼也靠不住。

    提示與參考解答：降取樣 4 倍後，新的取樣間距是 4 像素，依取樣定理能可靠解析的最短週期為 2 × 4 = 8 像素，因此週期 6 與 4 必然失真。週期 16 與 10 的新週期分別為 4 與 2.5 像素，高於臨界值，條紋保住。週期恰為 8 的情形最微妙：它的新週期正好是 2 像素，剛好踩在 Nyquist 頻率上。此時結果完全取決於取樣的相位——本例中每個取樣點都落在同一個相位（正弦的零點），於是整片變成均勻的灰色，條紋完全消失。這正是定理要求「高於兩倍」而非「至少兩倍」的原因：等於兩倍時，理論上資訊仍在，實務上卻可能完全取不到。

14. 計算題：承圖 3-8。環帶圖的瞬時徑向頻率為 r / 900 週期 / 像素。若降取樣 4 倍，從哪個半徑開始會出現混疊?若改為降取樣 2 倍呢?請說明這個半徑與「假環紋出現在外圈」的關係。

    提示與參考解答：降取樣 D 倍後的 Nyquist 頻率為 1 / (2D)。令 r / 900 = 1 / (2D) 得臨界半徑 r = 900 / (2D)。D = 4 時 r = 112.5 像素;D = 2 時 r = 225 像素。因為環帶圖的頻率由中心向外遞增，所以超過臨界半徑的區域才會混疊——這就是假環紋只出現在外圈的原因，中央完全不受影響。降取樣倍率愈小，臨界半徑愈往外推，受影響的範圍愈小。這張圖之所以是檢驗縮圖演算法的標準素材，正是因為「哪裡開始出錯」可以直接換算成「該演算法的有效截止頻率是多少」。

15. 計算題：承圖 3-7 的車輪。一個 8 輻的輪子以每秒 23 圈順時針旋轉，用 24 fps 拍攝。請計算每一影格之間輪子轉過的角度、以及「看起來」轉了多少，並說明為什麼看起來會倒轉。若輪子改為每秒 24 圈呢?

    提示與參考解答：每格之間輪子轉過 23 × 360 / 24 = 345 度。但 8 輻的輪子每轉過

> *360 / 8 = 45 度就與原本的樣子完全一樣，因此真正「看得出來」的只有 345 mod 45 = 30 度。又因為 30 度大於 45 / 2 = 22.5 度，大腦會選擇較短的那條路徑來解釋，也就是「往回轉 15 度」——於是看起來以每秒 15 × 24 / 360 = 1 圈的速度反向旋轉。若改為每秒 24 圈：每格轉過 360 度，360 mod 45 = 0，輪子看起來完全靜止。這與圖 3-6 中「週期恰為 8」的條紋消失是同一個數學現象——取樣頻率與訊號頻率成整數倍關係時，你什麼都取不到。*

16. 計算題：承圖 3-9 與 3-10。一張 8 位元影像中有一塊天空，其灰階標準差為 4.6。若量化到 4 位元（量化步長 16）與 6 位元（步長 4），分別會發生什麼？請以「標準差與量化步長的比值」說明判準，並據此推算這塊天空至少需要幾位元才不會出現假輪廓。

> *提示與參考解答：判準是把區域的灰階起伏與量化步長相比。4 位元時步長 16，遠大於起伏 4.6，整塊天空只會落在一兩個階上，階與階的交界形成生硬的假輪廓。6 位元時步長 4，小於起伏 4.6，漸變能被多個階層描述，假輪廓不明顯。一個實用的經驗法則是：量化步長應小於該區域的灰階標準差，最好小於其一半。要讓步長 ≤ 4.6 / 2 ≈ 2.3，需 256 / 2.3 ≈ 111 階，即至少 7 位元（128 階）。這解釋了為什麼天空、膚色漸層、柔和陰影是測試位元深度的最佳素材，也解釋了為什麼 HDR 影像需要 10 位元以上。*

17. 承圖 3-11：加了雜訊之後的均方根誤差反而變大，為什麼我們還說「抖動讓它看起來更好」？這與 3.5 節的哪一個論點是同一回事？請再舉一個「指標變差但觀感變好」的例子。

> *提示與參考解答：因為均方根誤差衡量的是逐像素的數值偏差，而人眼判斷平滑度靠的是「有沒有看得見的結構」。抖動把規則的階梯邊界打散成隨機顆粒：數值上偏差更大，但隨機顆粒在視覺上不構成結構，大腦會把它平均掉，規則的階梯卻會被視覺系統強化（馬赫帶效應）。這與 3.5 節的論點完全同源：指標衡量的面向不一定是你在乎的面向。另一個例子是適度的銳化（第 5 章）：它會讓 PSNR 下降（因為加入了原本沒有的過衝），卻普遍被認為看起來更清晰。底線是：指標用來比較同一類方法的優劣，不能用來跨類別排名，更不能取代目視確認。*

18. 承圖 3-13：拜爾陣列中綠色佔一半，為什麼？去馬賽克在什麼樣的影像內容上最容易出錯？這種錯誤與 3.2 節的混疊有什麼關係？

提示與參考解答：綠色佔一半有兩個理由：其一，人眼的視錐細胞對綠光（約 555 奈米）最敏感，綠通道對亮度感知的貢獻最大（第 8 章的亮度公式中綠的權重約 0.59）;其二，多一倍的綠色取樣點等於提高了亮度資訊的空間取樣率，而人眼對亮度細節遠比對色彩細節敏感。去馬賽克在「細密的高頻紋理」上最容易出錯，例如細格子襯衫、屋頂瓦片、遠處的欄杆，會產生原本不存在的彩色條紋（偽色）與拉鍊狀邊緣。關係很直接：每個顏色的實際取樣率只有影像解析度的二分之一到四分之一，當紋理頻率超過該色通道的 Nyquist 頻率時就會混疊——只是這次摺疊出來的不是假的亮度條紋，而是假的顏色。這也是為什麼高階相機會在感測器前加光學低通濾鏡。

19. 承圖 3-14：為什麼同樣提亮三倍，12 位元 RAW 的結果仍有兩百多個灰階，8 位元 JPEG 卻只剩七十幾個?請用數字說明，並解釋為什麼「先存成 JPEG 再後製」與「存 RAW 後製」在拍攝的那一刻看起來沒有差別。

    提示與參考解答：關鍵在量化發生的時機。12 位元有 4096 階，拍得過暗時訊號只佔其中低位的一小段，但那一段裡仍有數百個可用的階;提亮等於把這一段拉伸到整個 0 至 255，階層數足夠，拉伸後仍然平滑。8 位元只有 256 階，過暗的畫面可能只佔其中二三十階，提亮三倍等於把這二三十階拉開成橫跨整個範圍，相鄰階層之間出現空隙——這就是圖 3-14 右側直方圖呈梳齒狀的原因。至於拍攝當下看不出差別，是因為相機顯示的本來就是處理過的 8 位元預覽圖，兩者在那個階段確實一模一樣;差別要到你動手調整時才顯現。這也是 3.1.1 節「不可逆」最具體的一個例子。

20. 計算題：承 3.7.1 節。要掃描一份報紙，其最細的特徵是 6 點大小的鉛字，筆畫寬度約 0.1 毫米。(a) 依取樣定理，最低需要多少 dpi?(b) 若要能辨識而非只是偵測，實務上建議多少?(c) 若改設為 600 dpi，檔案會大幅增加，這些多出來的像素一定是浪費嗎?

    提示與參考解答：(a) 1 英寸 = 25.4 毫米。偵測 0.1 毫米的筆畫至少需每 0.05 毫米一個像素，即每英寸 25.4 / 0.05 ≈ 508 dpi——也就是說連「理論下限」都已經超過 300 dpi。(b) 實務上要三到五個像素涵蓋最小特徵，取四像素則每 0.025 毫米一像素，約 1016 dpi;若目標只是文字辨識（OCR），600 dpi 通常已足夠，因為 OCR 靠的是筆畫的整體形狀而非單一筆畫的精確寬度。(c) 不一定是浪費。多出來的像素在三種情況下有價值：要做 OCR 時更高的解析度能提升辨識率;原稿有網點（半色

調）時需要足夠解析度才能用第 6 章的陷波濾波移除;以及典藏用途——原稿會劣化，但掃描檔不會，一次掃到位比日後重掃便宜得多。真正的浪費是「原稿本身的細節有限卻硬提高解析度」，例如掃描一張已經模糊的老照片。

21. 承 3.7.2 節：醫學影像為什麼需要 12 至 16 位元?既然螢幕只能顯示 256 階、人眼也分辨不了那麼多，多出來的位元有什麼用?請說明「視窗化」的作用，並與第 5 章的對比強化比較。

    提示與參考解答：多出來的位元用在「儲存」而非「顯示」。人體組織之間的密度差異可能極小——早期腫瘤與正常組織的 CT 值可能只差幾個 Hounsfield 單位，若一開始就量化成 256 階，這個差異會被量化步長吃掉，資訊在存檔的那一刻就消失了（3.1.1 節的不可逆）。保留 12 至 16 位元，等於把「要看哪一段」的決定權留到判讀時。視窗化就是行使這個決定權：醫師設定視窗中心與視窗寬度，把 12 位元資料中關心的那一段（例如肺窗、骨窗、腦窗）線性映射到螢幕的 256 階，其餘部分壓成全黑或全白。與第 5 章對比強化的關係是：兩者在數學上都是灰階轉換，但起點不同——視窗化是從高位元深度的原始資料中「選一段來看」，資訊一直都在;而對一張已經是 8 位元的影像做對比拉伸，是在有限的階層上硬拉，必然伴隨假輪廓。這個對比值得記住：有沒有保住位元深度，決定了你是在「選擇」還是在「硬拉」。

22. 承圖 3-15 與 3-16：五種劣化的 PSNR 幾乎相同，但「整體加亮」與「抹掉一小塊」連 SSIM 的排名也不符合人的判斷。請說明為什麼，並提出至少兩種補救的評估做法。

    提示與參考解答：原因在於兩個指標都是「全域平均」。整體加亮改動了 100% 的像素但每個像素只差一點點;抹塊只改動不到 2% 的像素但那一塊被完全破壞。平均下來兩者的誤差總量相近，所以 PSNR 相近;SSIM 雖然考慮了結構，但它取的是所有局部視窗的平均，98% 完好的區域把那 2% 的災難稀釋掉了，於是反而給了抹塊更高的分數。補救做法至少有三種：其一，報告局部指標的最壞值而非平均值，例如 SSIM 差異圖的最小值、或誤差最大的前 1% 區域的平均誤差;其二，使用任務導向的指標——若目的是偵測瑕疵，就直接量偵出率與誤報率（第 16 章的 mAP 正是這種思路）;其三，永遠保留目視檢查，並把 SSIM 差異圖（圖 3-17）當成除錯工具，它會直接指出「哪裡不像」。一句話總結：指標是用來篩選候選方案的，不是用來下最終結論的。

23. 承圖 3-18：為什麼實務上前景與背景要採用互補的連通性?若兩者都用 8-連通，會出現什麼矛盾?請以「一條斜線把區域分成兩半」為例說明。

    提示與參考解答：考慮一條由左上到右下、只在對角相接的斜線，把一個白色區域分成左下與右上兩塊背景。若前景（線）與背景都用 8-連通：線本身是連通的（對角相接算相鄰），所以它「是一條完整的線」;但左下與右上的背景也因為對角相接而算連通，於是這條線「並沒有把區域分開」——一條完整的線卻沒有分割作用，這在拓撲上是矛盾的。反之若兩者都用 4-連通：線本身是斷開的一群孤立點，但背景被它分成了兩塊——一條斷掉的線卻有分割作用，同樣矛盾。採用互補的連通性（前景 8、背景 4，或前景 4、背景 8）可以消除矛盾，讓「連通」與「分離」這兩個概念自洽。這就是為什麼 OpenCV 的 connectedComponents 要明確指定 connectivity，而第 10 章的形態學運算也必須留意結構元素的形狀。

24. 承圖 3-19：同一個形狀用三種距離度量做距離轉換，最大值分別為 41.5、43.0、31.0。請解釋為什麼歐氏距離的值介於另外兩者之間，並說明在什麼情況下該選哪一種。

    提示與參考解答：三者的大小關係是固定的：D8 ≤ D2 ≤ D4。直覺上，棋盤距離（D8）允許斜走，一步可以同時前進一格橫向與一格縱向，所以步數最少、數值最小;城市街區距離（D4）只能上下左右走，斜向移動要拆成兩步，數值最大;歐氏距離是真正的直線距離，自然介於兩者之間。以一個 (3, 4) 的位移為例：D4 = 7、D2 = 5、D8 = 4，恰好印證這個順序。選用時機：要做實際的長度量測（例如量瑕疵的尺寸、第 10 章求內切圓半徑）必須用歐氏距離，因為只有它與真實世界的距離成正比;若目的只是快速估算或做形態學的膨脹腐蝕，D4 與 D8 的計算成本低得多（只需整數加法與比較），且 D8 恰好對應方形結構元素、D4 對應十字形結構元素，在實作上更自然。

25. (綜合設計題)某產線要以相機檢測印刷電路板上的焊點，最小焊點直徑 0.3 毫米，板子尺寸 200×150 毫米，產線每 2 秒通過一片，焊點的良與不良差別在於亮度分布而非形狀。請提出完整的取像規格（解析度、位元深度、曝光時間），每一項都要說明推算依據，並指出你最不確定的假設是什麼、要如何驗證。

    提示與參考解答：本題無標準答案，評分重點在推算鏈的完整性與自我檢驗。一組

合理的推算——解析度：焊點 0.3 毫米，取 5 像素涵蓋（因為要判斷亮度分布而非只偵測存在，需要較多取樣點），即每像素 0.06 毫米;水平需 200 / 0.06 ≈ 3333 像素、垂直 150 / 0.06 = 2500 像素，故選 4096×3000 左右的相機，或以兩台相機分區拍攝。位元深度：既然判準是亮度分布的細微差異，8 位元可能不足，建議 10 至 12 位元，並依習題 16 的判準實測——取一批良品與不良品，量測兩者亮度差的標準差，確認量化步長小於該差異的一半。曝光時間：每 2 秒一片，若板子是連續移動的，需由移動速度與可容忍的運動模糊（建議小於半個像素，即 0.03 毫米）反推曝光上限（第 7.6.3 節的做法）;若可停止取像則限制大幅放寬，但要確認機構的穩定時間。最不確定的假設通常有三個：「5 個像素夠不夠」（應以實拍的良品與不良品做 ROC 曲線驗證）、「打光是否穩定且均勻」（應量測同一片板子連拍數十張的亮度變異，若變異大於良與不良的差異，再高的解析度也沒用）、以及「板子是否會翹曲造成失焦」（應量測景深並實測邊角）。這題的核心訊息是：規格要從需求推算，但推算建立在假設上，而工程的專業在於知道自己假設了什麼、以及如何驗證它。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 2(數位影像基礎)。
- NIST, First Digital Image — Russell Kirsch, 1957:https://www.nist.gov/mathematics-statistics/first-digital-image(本章叫獸開講之史料來源)
- Wang, Z., Bovik, A. C., et al., Image Quality Assessment: From Error Visibility to Structural Similarity, IEEE TIP, 2004.(SSIM 原始論文)
- Shannon, C. E., Communication in the Presence of Noise, Proc. IRE, 1949.(取樣定理)
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 2(影像形成與感測器)。
- scikit-image 官方文件(SSIM 等品質指標):https://scikit-image.org

- 補充教材與課程資訊：@prof(https://page.line.me/prof)